

Different Terms

- **Command Line** - any interface that is used by entering textual commands (gen. **Windows** centric)
- **Terminal** - This is a type of command line (gen. **Mac** centric)
- **Console** - A command-line interface used to work with your computer
- **Shell** - A program running on terminal
- **Bash** - A popular shell on Mac OS/ Linux
- **Z-Shell** - Another shell (default)



Some basic git bash commands



Initialize Git

</> Bash

```
git init          # Initialize a Git repository
git clone <url>    # Clone a repository
git status        # Check repository status
```



Navigation

</> Bash

```
pwd              # Show current directory
ls               # List files
ls -a           # Show hidden files
cd folder       # Enter folder
cd ..           # Go back one directory
cd ~            # Go to home directory
mkdir name      # Create a folder
rm file.txt     # Delete a file
rm -r folder    # Delete a folder
clear           # Clear terminal
```



Staging Files

</> Bash

```
git add file.txt      # Stage one file
git add .             # Stage everything
git restore file.txt  # Discard changes in a file
git restore --staged file.txt # Unstage a file
```



Committing

</> Bash

```
git commit -m "Initial commit"
git commit -am "Update files"  # Stage tracked files and commit
git log                       # View commit history
git log --oneline             # Short history
```



Branches

</> Bash

```
git branch                # Show branches
git branch feature        # Create branch
git checkout feature      # Switch branch
git switch feature        # Modern way to switch
git switch -c feature     # Create and switch
git merge feature         # Merge feature into current branch
git branch -d feature     # Delete branch
```



Remote Repository

</> Bash

```
git remote -v
git remote add origin <url>
git push -u origin main
git push
git pull
git fetch
```



View Changes

</> Bash

```
git diff           # Unstaged changes
git diff --staged  # Staged changes
git show           # Show last commit
```



Undo

</> Bash

```
git reset HEAD file.txt      # Unstage file
git reset --soft HEAD~1     # Undo commit, keep changes staged
git reset --mixed HEAD~1    # Undo commit, keep changes unstaged
git reset --hard HEAD~1     # Delete last commit completely
git checkout -- file.txt    # Restore file (older command)
git restore file.txt        # Restore file (recommended)
```



Tags

</> Bash

```
git tag
git tag v1.0
git push origin v1.0
```



Configure Git

⌘ Bash

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

git config --list
```



SSH

⌘ Bash

```
ssh-keygen -t ed25519 -C "you@example.com"
cat ~/.ssh/id_ed25519.pub
ssh -T git@github.com
```



Stash

⌘ Bash

```
git stash
git stash list
git stash pop
git stash apply
```

Clean

 Bash

```
git clean -fd      # Remove untracked files/folders
```

Typical Workflow

 Bash

```
git clone <repo-url>
```

```
cd project
```

```
git status
```

```
git add .
```

```
git commit -m "Added new feature"
```

```
git push
```

★ Commands You'll Use the Most

⌞ Bash

```
git status
```

```
git add .
```

```
git commit -m "message"
```

```
git push
```

```
git pull
```

```
git clone <url>
```

```
git log --oneline
```

```
git branch
```

```
git switch branch-name
```

```
git merge branch-name
```

```
git diff
```

```
git stash
```

```
git fetch
```

```
git remote -v
```


Paths

Absolute & Relative

cd Desktop/Delta (relative path)

cd /Users/shradhakhapra/Desktop (absolute path)

/ = root directory

~ = home directory

mkdir : to create a directory

Flags

Flags are characters that we pass with commands to modify their behaviour

manual Command

man ls - give info about ls command

man mkdir - give info about mkdir command

with Flags

`ls -l`

`ls -a`

`ls -la`

Flags (also called options or switches) modify the behavior of a command. They usually start with `-` (short flag) or `--` (long flag).

General Syntax

```
</> Bash
command -f
command -a -b
command -ab
command --long-flag
command --flag=value
```



Example:

```
</> Bash
ls -a
git log --oneline
```



Common Terminal Flags

ls (List files)

Flag	Meaning
-a	Show hidden files
-l	Long listing format
-h	Human-readable sizes
-R	Recursive listing
-t	Sort by modification time
-r	Reverse order

Examples:

❏ Bash

```
ls -a
ls -lh
ls -la
ls -ltr
```



Most Frequently Used Flags

Flag	Meaning	
<code>-a</code>	All / hidden / auto-stage (depends on command)	
<code>-A</code>	All changes	
<code>-r</code>	Recursive	
<code>-R</code>	Recursive (capital form in some commands)	
<code>-f</code>	Force	
<code>-i</code>	Interactive / confirm	
<code>-v</code>	Verbose	
<code>-h</code>	Human-readable or help (depends on command)	
<code>-l</code>	Long format	
<code>-n</code>	Number or limit	
<code>-p</code>	Patch / parents (depends on command)	
<code>-m</code>	Message	
<code>-u</code>	Upstream / update	
<code>-s</code>	Short / silent (depends on command)	

Important: The same flag can mean different things for different commands. For example, `-a` means "show hidden files" for `ls`, but "auto-stage tracked files" for `git commit`. Always check `command --help` or `man command` for the exact meaning.



Help Flag (Almost Every Command)

```
<> Bash
command --help
```



Examples:

```
<> Bash
git --help
git status --help
ls --help
grep --help
```



or

```
<> Bash
man ls
man git
```



git branch

Flag

Meaning

-a

All branches

-d

Delete branch

-D

Force delete

Examples:

⌘ Bash

```
git branch -a  
git branch -d feature
```

git checkout

Flag

Meaning

-b

Create and switch branch

Example:

⌘ Bash

```
git checkout -b feature
```



Git Flags

`git status`

Flag	Meaning
<code>-s</code>	Short output
<code>-b</code>	Show branch

Example:

`</> Bash`

```
git status -sb
```

git log

Flag	Meaning
<code>--oneline</code>	One commit per line
<code>--graph</code>	ASCII commit graph
<code>--all</code>	Show all branches
<code>-p</code>	Show patch/diff
<code>-n 5</code>	Last 5 commits

Examples:

⌵ Bash

```
git log --oneline
git log --graph --oneline --all
git log -n 10
```

git add

⌘ Bash

```
git add .  
git add -A  
git add -u
```

Flag	Meaning
.	Current directory changes
-A	All changes (tracked + untracked)
-u	Only tracked files

git commit

Flag	Meaning
-m	Commit message
-a	Stage tracked files automatically
--amend	Modify last commit

Examples:

⌘ Bash

```
git commit -m "Fix bug"  
git commit --amend
```


git push

Flag	Meaning
<code>-u</code>	Set upstream branch
<code>--force</code>	Force push
<code>--tags</code>	Push tags

Example:

`</>` Bash

```
git push -u origin main
```

1. touch Command

The `touch` command is mainly used to **create an empty file** or **update the timestamp** of an existing file.

Create a new file

`</>` Bash

```
touch file.txt
```



Creates `file.txt` if it doesn't already exist.

Create multiple files

`</>` Bash

```
touch a.txt b.txt c.cpp notes.md
```



Update timestamp

If the file already exists:

`</>` Bash

```
touch file.txt
```



This updates its last modified time.

2. Deleting Files & Folders

Delete a file

```
</> Bash  
rm file.txt
```



Delete multiple files

```
</> Bash  
rm a.txt b.txt c.txt
```



Delete an empty folder

```
</> Bash  
rmdir folder
```



Delete a folder with all its contents

```
</> Bash  
rm -r folder
```



Force delete (no confirmation)

```
</> Bash  
rm -rf folder
```



- `-r` → Recursive (delete folders and everything inside)
- `-f` → Force (don't ask for confirmation)



⚠ Be careful with `rm -rf`. Deleted files usually cannot be recovered easily.

Common file commands

Command	Purpose
<code>touch file.txt</code>	Create an empty file
<code>mkdir folder</code>	Create a folder
<code>ls</code>	List files
<code>pwd</code>	Show current directory
<code>cd folder</code>	Enter a folder
<code>rm file.txt</code>	Delete a file
<code>rmdir folder</code>	Delete an empty folder
<code>rm -r folder</code>	Delete a folder recursively
<code>rm -rf folder</code>	Force delete a folder and its contents